

Sys-28 Send Notification Testing System Requirements

Needed to install Nodemailer and Schedule:

```
cd backend
npm i nodemailer
npm i -D @types/nodemailer
npm i @nestjs/schedule
npm i class-validator class-transformer
```

To Access the Supabase:

Need to make the file '.env' under the backend folder and copy and paste this:

```
SUPABASE_URL=https://cnzlgtpnlxkvzurmpjsu.supabase.co
SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6ImNuemxndHBubHhrcnplcm1wanNlIiwicm9sZSI6ImFub24iLCJpYXQiOiE3NzE2OTgyOTEsImV4cCI6MjA4NzI3NDI5MX0.85vhT11dBNk5_vcBJE6OV83IsyrLVFDtvrfOGdPOiLY
RECAPTCHA_SECRET_KEY=6LcZ0QwsAAAAAEo_yk4A5NSlJZpvh6ictaPhktNA
```

RECAPTCHA:

Create .env.local file under frontend.

Copy and paste this into .env.local:

```
NEXT_PUBLIC_RECAPTCHA_SITE_KEY=6LcZ0QwsAAAAAJ3Bhrh0Yv140BqyU5sdihlPFYgR
BACKEND_URL=http://localhost:3000
NEXT_PUBLIC_APP_URL=http://localhost:3001
```

How to Run:

BACKEND

```
1] cd backend
2] npm install
3] npm run start:dev
```

Backend is running on http://localhost:3001

FRONTEND

```
1] cd frontend
2] npm install
3] npm run dev -p 3000
```

Sys-28 Summary

Files Modified:

- backend/package-lock.json
- backend/package.json
- backend/src/app.module.ts

Files Created:

- backend/src/notifications/email.service.ts
- backend/src/notifications/notifications.controller.ts
- backend/src/notifications/notifications.module.ts
- backend/src/notifications/notifications.service.ts
- backend/src/notifications/preferences.service.ts
- backend/src/notifications/reminder.processor.ts
- backend/src/notifications/reminders.controller.ts
- backend/src/notifications/reminders.service.ts
- backend/src/notifications/templates/followup.template.ts

Achievements:

Implements follow-up reminder scheduling and notification delivery.

Features:

- POST /reminders/followup to schedule follow-ups
- Background processing
- Dashboard notifications created for due reminders
- Reminder lifecycle: SCHEDULED → SENDING → COMPLETED
- Notification template includes:
 - Client details
 - Interaction details
 - Last interaction timestamp
 - Follow-up interval and due date

Notes:

- The follow-up interval is set by the user. For testing purposes, the starting date should be set to (last interaction timestamp - followup interval). For example, if the follow-up interval = 2 days, for testing, the last interaction time stamp should have 2 days subtracted off of it in order to show the functionality immediately.
- The reminder life cycle is currently scheduled to run every 10 seconds for testing purposes, after implementation and connection with the workers' database, the interval should be set to cycle every 60 seconds (or 1 minute).

Sys-28 Testing

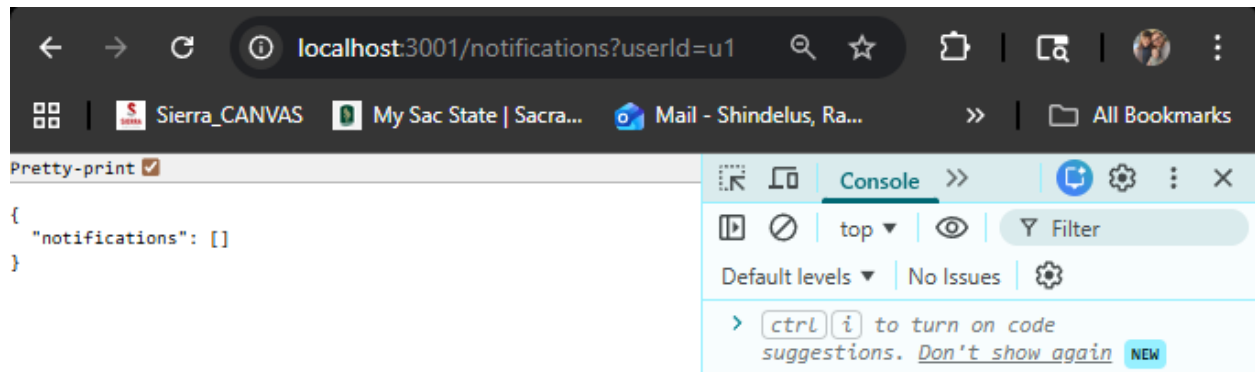
*Using Javascript that is copied and pasted into the browsers' console. Visiting <http://localhost:3001/notifications?userId=u1> and pressing F12 on the keyboard. Then selecting 'console' and pasting various inputs for testing purposes.

Immediate Reminder Testing

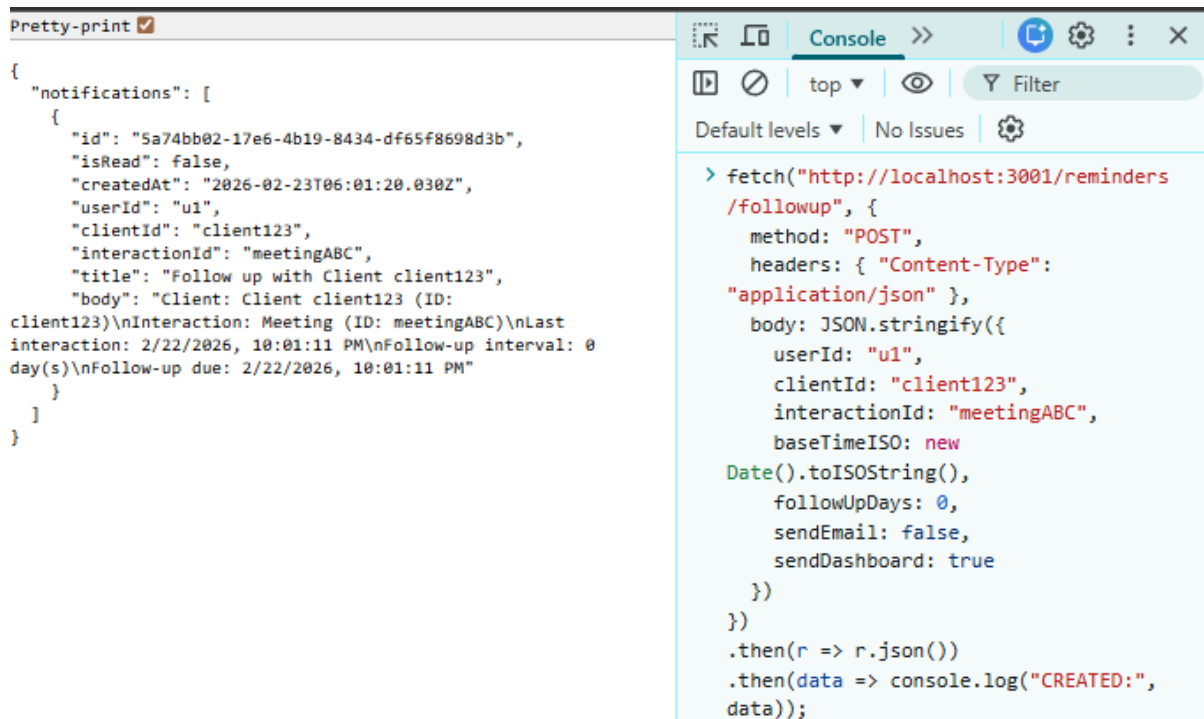
- Shows that the reminder is processed within 10 seconds
 - The reminder is first created (currently through console input)
 - The notification is then created
 - The reminder is sent and marked completed

Prior to entering JavaScript into the console:

- Show no (empty) notifications



After providing JavaScript to the console:



2-Day Follow-Up Testing

- This has been backdated to allow for immediate processing to show the functionality.
- The baseTimeISO has been set to two days prior, so that the two day follow up period (followUpDays = 2) may be shown immediately.
- This also shows multiple notifications being allowed at once (there are two separate ones shown on the screen)

```

Pretty-print ☒
{
  "notifications": [
    {
      "id": "5a74bb02-17e6-4b19-8434-df65f8698d3b",
      "isRead": false,
      "createdAt": "2026-02-23T06:01:20.030Z",
      "userId": "u1",
      "clientId": "client123",
      "interactionId": "meetingABC",
      "title": "Follow up with Client client123",
      "body": "Client: Client client123 (ID: client123)\nInteraction: Meeting (ID: meetingABC)\nLast interaction: 2/22/2026, 10:01:11 PM\nFollow-up interval: 0 day(s)\nFollow-up due: 2/22/2026, 10:01:11 PM"
    },
    {
      "id": "bc5511f6-cf71-4ebd-90aa-c2dde79bf728",
      "isRead": false,
      "createdAt": "2026-02-23T06:06:00.018Z",
      "userId": "u1",
      "clientId": "enterpriseClient",
      "interactionId": "consultation42",
      "title": "Follow up with Client enterpriseClient",
      "body": "Client: Client enterpriseClient (ID: enterpriseClient)\nInteraction: Meeting (ID: consultation42)\nLast interaction: 2/20/2026, 10:05:56 PM\nFollow-up interval: 2 day(s)\nFollow-up due: 2/22/2026, 10:05:56 PM"
    }
  ]
}

```

```

> const twoDaysAgo = new
  Date(Date.now() - 2 * 24 * 60 * 60 *
    1000).toISOString();

fetch("http://localhost:3001/reminders/followup", {
  method: "POST",
  headers: { "Content-Type":
    "application/json" },
  body: JSON.stringify({
    userId: "u1",
    clientId: "enterpriseClient",
    interactionId: "consultation42",
    baseTimeISO: twoDaysAgo,
    followUpDays: 2,
    sendEmail: false,
    sendDashboard: true
  })
})
.then(r => r.json())
.then(data => console.log("CREATED:",
  data));

< ▶ Promise {<pending>}
CREATED: ▶ {reminder: {...}} VM4522:17

```

Future Reminders Testing

- This will prove that the scheduling logic is functioning as intended.
- Status shows 'SCHEDULED' without any new notifications

```

Pretty-print ☒
{
  "notifications": []
}

```

```

> fetch("http://localhost:3001/reminders/followup", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    userId: "u1",
    clientId: "futureClient",
    interactionId: "planningMeeting",
    baseTimeISO: new Date().toISOString(),
    followUpDays: 3,
    sendEmail: false,
    sendDashboard: true
  })
})
.then(r => r.json())
.then(data => console.log("CREATED:", data));

```

```

< Promise {<pending>}
  [[Prototype]]: Promise
  [[PromiseState]]: "fulfilled"
  [[PromiseResult]]: undefined

```

```

CREATED: VM4967:15
  {reminder: {...}}
    reminder: {id: '8e310b68-aadf-407', ...}
    [[Prototype]]: Object

```

Multiple Reminders Testing

- This will show that there can be multiple reminders and notifications at the same time without concurrency issues.
- Three notifications are created and continue existing simultaneously

```

Pretty-print
{
  "notifications": [
    {
      "id": "44bdd8f8-7348-4c82-b4e5-c22e28aab5e7",
      "isRead": false,
      "createdAt": "2026-02-23T06:14:10.011Z",
      "userId": "u1",
      "clientId": "bulkClient",
      "interactionId": "meeting0",
      "title": "Follow up with Client bulkClient",
      "body": "Client: Client bulkClient (ID: bulkClient)\nInteraction: Meeting (ID: meeting0)\nLast interaction: 2/22/2026, 10:14:09 PM\nFollow-up interval: 0 day(s)\nFollow-up due: 2/22/2026, 10:14:09 PM"
    },
    {
      "id": "3a05ab32-fd58-4037-9ec9-eea2f60c488d",
      "isRead": false,
      "createdAt": "2026-02-23T06:14:10.012Z",
      "userId": "u1",
      "clientId": "bulkClient",
      "interactionId": "meeting1",
      "title": "Follow up with Client bulkClient",
      "body": "Client: Client bulkClient (ID: bulkClient)\nInteraction: Meeting (ID: meeting1)\nLast interaction: 2/22/2026, 10:14:09 PM\nFollow-up interval: 0 day(s)\nFollow-up due: 2/22/2026, 10:14:09 PM"
    },
    {
      "id": "df38bad7-f0cf-48ff-a68d-38cdf55fe9d9",
      "isRead": false,
      "createdAt": "2026-02-23T06:14:10.012Z",
      "userId": "u1",
      "clientId": "bulkClient",
      "interactionId": "meeting2",
      "title": "Follow up with Client bulkClient",
      "body": "Client: Client bulkClient (ID: bulkClient)\nInteraction: Meeting (ID: meeting2)\nLast interaction: 2/22/2026, 10:14:09 PM\nFollow-up interval: 0 day(s)\nFollow-up due: 2/22/2026, 10:14:09 PM"
    }
  ]
}

```

```

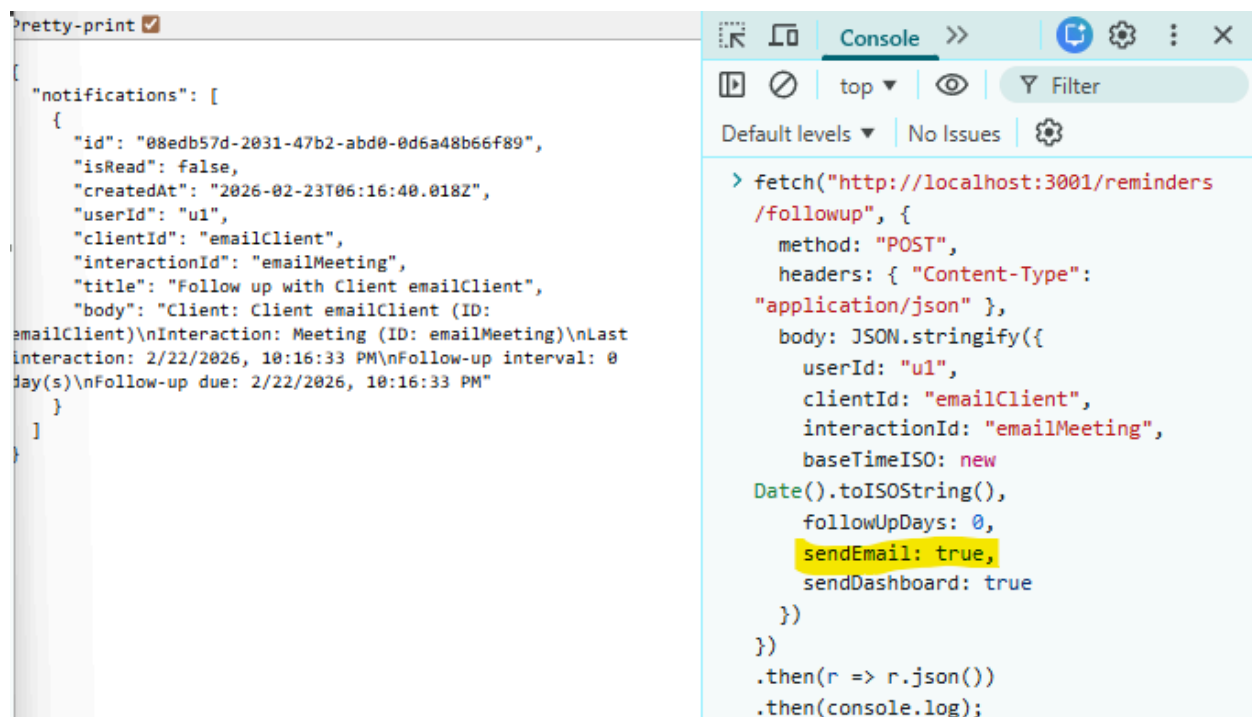
> for (let i = 0; i < 3; i++) {
  fetch("http://localhost:3001/reminders/followup", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      userId: "u1",
      clientId: "bulkClient",
      interactionId: "meeting" + i,
      baseTimeISO: new Date().toISOString(),
      followUpDays: 0,
      sendEmail: false,
      sendDashboard: true
    })
  });
}

```

Email Notification Sent Testing

- This testing shows that with proper SMTP set up, we can provide email notifications to clients and staff for follow-up meetings.

```
[Nest] 13864 - 02/22/2026, 10:16:40 PM    WARN [EmailService] SMTP not configured; would email test@example.com: Follow up with Client emailClient
Client: Client emailClient (ID: emailClient)
Interaction: Meeting (ID: emailMeeting)
Last interaction: 2/22/2026, 10:16:33 PM
Follow-up interval: 0 day(s)
Follow-up due: 2/22/2026, 10:16:33 PM
```



The screenshot displays a web application interface with two main panels. The left panel, titled 'Pretty-print', shows a JSON object representing a notification. The right panel, titled 'Console', shows a REST client request for a POST endpoint.

```
pretty-print ☒
{
  "notifications": [
    {
      "id": "08edb57d-2031-47b2-abd0-0d6a48b66f89",
      "isRead": false,
      "createdAt": "2026-02-23T06:16:40.018Z",
      "userId": "u1",
      "clientId": "emailClient",
      "interactionId": "emailMeeting",
      "title": "Follow up with Client emailClient",
      "body": "Client: Client emailClient (ID: emailClient)\nInteraction: Meeting (ID: emailMeeting)\nLast interaction: 2/22/2026, 10:16:33 PM\nFollow-up interval: 0 day(s)\nFollow-up due: 2/22/2026, 10:16:33 PM"
    }
  ]
}
```

The right panel shows a REST client request for a POST endpoint:

```
> fetch("http://localhost:3001/reminders/followup", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    userId: "u1",
    clientId: "emailClient",
    interactionId: "emailMeeting",
    baseTimeISO: new Date().toISOString(),
    followUpDays: 0,
    sendEmail: true,
    sendDashboard: true
  })
})
.then(r => r.json())
.then(console.log);
```